Localizing INT8 Quantization Collapse in EfficientNet-Bo

Edge poultry disease detection on Raspberry Pi 5

Fatih Jawwad Al Mumtaz
Amirul Al Aziz Muhammad Iqbal Hidayatullah
Universitas Darussalam Gontor — ICAIDES 2026 (ID 1571359798)

NOVEMBER 2026

YOUR LOGO HERE

SECTION 1

Why edge, why this benchmark

Poultry disease costs; lab diagnosis out of reach; the deploy gap.

Farm-level detection needs models that run on a Pi, not a GPU.

Poultry disease (Coccidiosis, Salmonellosis, Newcastle) causes major losses, yet lab diagnosis is out of reach for small farms. Prior work hits **99.12%** (EfficientNet-B0) and **97.1%** (MobileNetV2) — but targets the cloud, not the edge.

Two questions drive this work: which lightweight architectures survive `INT8` on CPU, and what does the accuracy-latency tradeoff look like on a **Raspberry Pi 5**?

Scope: three models, one board, one runtime (ORT 1.29.0). No claimed generality beyond the measured stack.

8,770 published images — audited, then deduplicated to 8,153.

claimed

8,770 leak-free images (FecalFed benchmark)

→

audit

pHash audit: 617 near-duplicate pairs, 94 test images byte-identical to train

→

used

sterile 8,153 split, test n = 1,728, seed 42

All reported numbers use the deduplicated split. Four classes: Coccidiosis, Healthy, Salmonella, Newcastle (NCD underrepresented: 141 test).

SECTION 2

What survives INT8, and what collapses

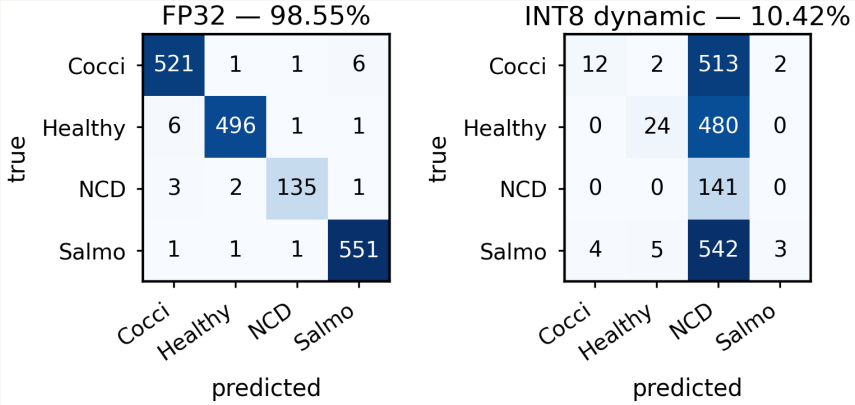
Dynamic INT8 keeps two models, destroys the third.

Dynamic INT8: two survive, one collapses

	MNV2	SNV2	EB0
FP32	97.92%	97.69%	98.55%
INT8 dyn	97.28%	97.63%	**10.42%**
Drop	0.64 pp	0.06 pp	**88.13 pp**

Paired McNemar on the same 1,728 test images: MNv2 discordant (b=18, c=7, p=0.0455), SNv2 not (b=6, c=5, p=1.0). EB0 degenerates to a **near-single-class predictor** — 1,676 of 1,728 land in the Newcastle column.

Where exactly EBo fails



FP32 (left) vs dynamic INT8 (right). Per-class INT8 recall: cocci 2.27%, healthy 4.76%, salmo 0.54%, NCD 100%. Matrices for MNv2/SNv2 in the repo.

Per-channel recovers it: 9.32% vs 94.56%

	ACC	SIZE	PI T=4
dynamic	10.42%	4.16 MB	63.99 ms
static per-tensor	9.32%	4.29 MB	13.63 ms
static per-channel	**94.56%**	4.67 MB	**13.50 ms**

Per-channel vs dynamic, paired: b=14, c=1468, **$p < 1e\text{-}15$**. Against FP32 the gap is still real (b=75, c=7, $p \sim 1e\text{-}13$) — **recovery, not parity**. Balanced 25/class replication on a retrained model: per-tensor 27.14% vs per-channel 93.40%.

Depthwise per-channel alone recovers the model; without it, collapse.

	CONFIG	ACC	COLLAPSE
C depthwise per-ch, rest per-tensor	4.33 MB	**93.46%**	no
D all per-ch except depthwise	4.51 MB	8.33%	yes
E depthwise left in FP32	4.69 MB	97.16%	no
F SE left in FP32	5.91 MB	12.44%	yes

Mechanism: shared per-tensor scale is set by the largest channel; depthwise channel ranges spread up to **112x** (median P95/P5 39.94x), worst channels quantize to a constant (0 dB). Prior work flagged this family (Sheng 2018); our contribution is the localized case study + runtime behavior.

SECTION 3

What the Pi actually does

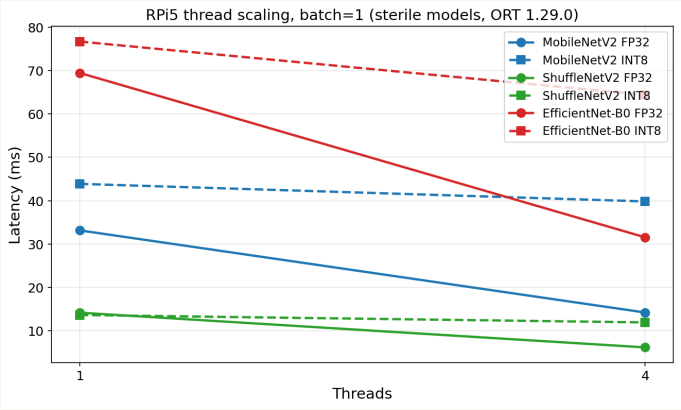
Dynamic INT8 slows down; static per-channel speeds up 2.34x.

On the Pi: dynamic INT8 is slower than FP32

	FP32 T=4	INT8 T=4	SCALING	SLOWDOWN
MNv2	14.23 ms	39.83 ms	1.10x	2.80x
SNv2	6.19 ms	11.95 ms	1.14x	1.93x
EB0	31.56 ms	64.39 ms	1.19x	2.04x

FP32 scales 2.20–2.33x from 1 to 4 threads; INT8 manages 1.10–1.19x. Kernel evidence: each Conv expands into DynamicQuantizeLinear + ConvInteger + Mul + Cast (1.37–3.05x node growth), quantization kernels add 20.7–29.2% overhead.

Static flips it: 2.34x faster than FP32



EB0 static per-channel: 13.50 ms vs FP32 31.53 ms at 4 threads. Same board, same runtime, separate session within 0.7% of the sweep. Static is faster AND more accurate here — but one board, one runtime, one seed.

So which model do you ship?

fastest
SNv2 FP32: 6.19 ms, 97.69%

→

balanced
MNv2 FP32: 14.23 ms, 97.92%

→

accurate
EB0 static per-ch: 13.50 ms, 94.56%

Best and worst FP32 differ by 0.86 pp — let hardware and scheme decide. 3.3–3.7x size shrink with INT8; RAM at inference barely moves, so do not claim memory savings on this stack.

Limitation, stated plainly: single Coccidiosis-only calibration slice; one seed; energy not measured (current telemetry null).

SECTION 4

Close

Deployment depends on architecture, runtime, and granularity jointly.

Takeaway and artifact

INT8 is not one thing: scheme × granularity × runtime decides.

- **Dynamic INT8** preserves MNv2/SNv2, collapses EB0 to 10.42%
- **Per-channel static** recovers EB0 to 94.56%, 2.34x faster than FP32
- **Depthwise granularity** is the localized cause (93.46% vs 8.33%)
- Artifact: `github.com/jaweed3/paper_poultry_p2` + Zenodo 10.5281/zenodo.23374216

Thank you. Questions welcome — the failure modes are in the paper.